



CHALLENGE 07 / 20

THE FILELESS INVOICE

EXECUTION RECONSTRUCTION WITH MINIMAL DISK EVIDENCE

NATIONAL LEVEL HACKATHON • 2026

POWERED BY SYNTHOQUEST PVT LTD

Student Project Development Challenge • No organizer-provided evidence package.

1 | PROBLEM BACKGROUND

An accounts payable clerk at a power distribution company with 1,900 staff opened an invoice at 14:22. By 14:40 her workstation was querying the directory, file servers and, through a jump host, a plant historian. She rebooted at 14:55. That night a scheduled patch cycle ran. No memory was captured and no sample of the code was kept.

What remains is sparse: DNS and proxy logs, authentication events, EDR events sampled at one in twenty, default Windows event logs and network flow summaries. Much of what the attacker did used tools that administrators also use every day.

The energy regulator requires proof of what was reached on the plant side, and the company cannot say.

2 | PROBLEM STATEMENT

Develop an **Execution Inference System** that reconstructs what code ran, which capabilities it exercised, which credentials and data it touched and which downstream systems it reached, when the code and its memory no longer exist and the surviving evidence is sparse, sampled and partly benign-looking.

The system must bound what can never be known and state which regulatory questions it can and cannot answer.

Hard constraints

- There is no sample, no memory image and no full log; evidence is sampled and incomplete.
- Activity indistinguishable from routine administration must be separated from the attacker's.
- Downstream access must be attributed to this execution or to unrelated activity.
- Each inferred action needs a stated confidence that the team's own tests show to be calibrated.
- Teams must use benign stand-ins for attacker behaviour. No functional malware may be created or used.

No evidence package or dataset is provided. Teams decide what to build, what data they need, how to simulate the environment and how to validate the result. No tools or architecture are prescribed.

3 | MANDATORY FEATURES

Teams must develop a working software solution. The system must provide the following features. How each is achieved is for the team to decide.

01	Sparse-Evidence Reasoning	Ability to reason from sampled, partial and ambiguous traces.
02	Execution Reconstruction	Ability to infer what executed without the code itself.
03	Capability and Intent Inference	Ability to infer which capabilities were exercised.
04	Routine-Activity Separation	Ability to separate attacker use of normal tools from administrators'.
05	Credential and Data Exposure Inference	Ability to infer what was touched or taken.
06	Downstream Attribution	Ability to attribute later access to this execution or to something else.
07	Knowable-Limit Analysis	Ability to state what can never be recovered.
08	Regulatory Question Mapping	Ability to state which regulatory questions can be answered.

The system must clearly distinguish between:

OBSERVED	INFERRED	PLAUSIBLE	HYPOTHESIS	UNRECOVERABLE
----------	----------	-----------	------------	---------------

An inferred action that cannot be supported must not be presented as having occurred.

Build, don't search. This is a development challenge. Teams must create the system and the controlled environment that proves it works. Submitting an analysis of a given dataset is not a solution.

4 | EXPECTED OUTPUT

Live demonstration. The team must show that its system can handle a complex controlled scenario created by the team. The demonstration must show:

01	Teams' own simulated utility environment with benign stand-in behaviour
02	Evidence sampled and degraded as in the scenario
03	Reconstruction of actions and order
04	Separation from routine administration
05	Downstream access attribution
06	Calibrated confidence per inferred action
07	The list of questions that cannot be answered

The team must be able to explain why the system reached each major conclusion.

Validation standard (every team)

- Build a scenario generator with known ground truth and evaluate on at least ten distinct scenarios, including ones with deliberately misleading evidence.
- At the demonstration, judges change scenario parameters on the spot. The system must adapt without being rebuilt.
- Include an adversary variant that actively tries to defeat the system, and report how the system held up.
- Stated confidence must match measured accuracy. Overconfident or unexplained conclusions lose marks.
- No real systems, real personal data or functional malware may be used.

Submission. Each team must submit:

01	Working Prototype	A functional implementation of the proposed solution.
02	Source Code	Complete source code required to reproduce the project.
03	Technical Architecture	Architecture and component description.
04	Validation Environment	A controlled environment created by the team to demonstrate the solution.
05	Technical Documentation	Implementation, design and testing documentation.
06	Forensic Investigation Report	A professional report generated from the demonstrated investigation.
07	Demonstration	Live technical demonstration to the judging panel.
08	Limitations	Known limitations and situations where the system cannot establish a reliable conclusion.